Guía de pruebas en Postman

TPI Logística de última milla · Backend de Aplicaciones 2026 · Comisión 3K4

Esta guía explica cómo levantar el sistema completo en local y probarlo con la colección de Postman del repositorio (`postman/`). La colección recorre el proceso de punta a punta, los escenarios alternativos (cancelación, entrega fallida, reprogramación, pedido no asignable) y las pruebas de seguridad, y verifica cada respuesta automáticamente. Al final están los escenarios que se prueban a mano: caída de servicios, OSRM y mails.

Todas las requests van contra el **api-gateway** (`http://localhost:8080`). Nunca se llama directo a los microservicios: así se prueba también el ruteo y la validación del token.

1. Requisitos

Herramienta	Versión	Para qué
Postman (app de escritorio)	cualquiera reciente	Ejecutar la colección. La versión web necesita el Postman Agent para llegar a localhost.
JDK	21 o superior	Compilar y correr los servicios.
Maven	3.9 o superior	`mvn spring-boot:run` y `mvn install`.
PostgreSQL	17 o superior	Una base por servicio: `pedidos_db` y `despacho_db`.
Keycloak	26	Login y emisión de tokens. Se descarga como .zip desde keycloak.org y se descomprime.

Todas las credenciales y URLs (bases de datos, Keycloak, OSRM y Gmail) ya están cargadas en los `application.yml` de cada servicio: no hace falta configurar nada para correrlo. Cualquier valor se puede sobrescribir con una variable de entorno, sin tocar el archivo:

```
setx OSRM_URL "http://localhost:5000"
```

> **Importante.** después de un `setx` hay que cerrar y volver a abrir la terminal (o IntelliJ) para que la variable se vea, y reiniciar el servicio.

2. Levantar el sistema

El orden importa: cada servicio necesita que lo anterior ya esté corriendo. Cada comando va en su propia terminal, que queda abierta mientras se prueba.

Paso 1 · Bases de datos (solo la primera vez)

Con PostgreSQL corriendo, desde la carpeta del repositorio:

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -f infra/init-db.sql
```

(Cambiar `18` por la versión instalada.) Crea las dos bases con sus usuarios. Las tablas las crea cada servicio al arrancar.

Paso 2 · Keycloak

Copiar `infra/realm-logistica.json` a la carpeta `data/import` de Keycloak (crearla si no existe) y arrancarlo **desde la carpeta de Keycloak**:

```
cd C:\ruta\a\keycloak-26.x.x
.\bin\kc.bat start-dev --http-port=8180 --import-realm
```

Está listo cuando aparece `Listening on: http://0.0.0.0:8180`. El realm `logistica` trae los roles, los clients y los usuarios de prueba.

Paso 3 · Librería común (la primera vez y cada vez que cambie)

```
cd logistica-commons
mvn clean install
```

Paso 4 · Microservicios, en este orden

```
cd pedidos-service   ;  mvn spring-boot:run     # puerto 8081
cd despacho-service  ;  mvn spring-boot:run     # puerto 8082
cd api-gateway       ;  mvn spring-boot:run     # puerto 8080
```

Cada uno está listo cuando muestra `Started ...Application`. También se pueden correr desde IntelliJ con la clase `*Application` de cada proyecto.

Componente	Puerto	Comprobación rápida en el navegador
Keycloak	8180	`http://localhost:8180/realms/logistica` devuelve un JSON
pedidos-service	8081	`http://localhost:8081/swagger-ui.html`
despacho-service	8082	`http://localhost:8082/swagger-ui.html`
api-gateway	8080	`http://localhost:8080/actuator/health` devuelve UP

> **Datos iniciales.** despacho-service carga solo, al arrancar, los datos de prueba: 2 centros (1 = Córdoba, 2 = Neuquén), 4 vehículos y 3 repartidores. No hace falta crearlos a mano.

3. Correr la colección

La colección prueba el sistema completo sola: pide los tokens, crea los datos, recorre el proceso de punta a punta y verifica cada respuesta con `pm.test`. No hay que copiar ids ni tokens a mano.

Opción A · Desde Postman

- **Import** (arriba a la izquierda) y arrastrar los dos archivos de la carpeta `postman/`: `TPI-Logistica.postman_collection.json` y `Local.postman_environment.json`.
- Arriba a la derecha, elegir el entorno **Local**.
- Sobre la colección **TPI Logística**: **Run collection** → **Run TPI Logística**. Corre las 53 requests en orden.

Opción B · Desde la consola (Postman CLI)

```
postman collection run postman/TPI-Logistica.postman_collection.json `
    -e postman/Local.postman_environment.json
```

Resultado esperado: 53 requests y 91 verificaciones, sin fallas. Tarda unos 15 segundos.

> **Una vez por día.** un vehículo y un repartidor hacen una sola ruta por día (RN11). Para correr la colección otra vez el mismo día, primero hay que volver los datos al estado inicial (sección 5).

Si se prefiere ir paso a paso, se puede ejecutar cada request con **Send** respetando el orden de las carpetas: cada una guarda en el entorno los ids y tokens que usan las siguientes.

4. Qué verifica cada carpeta

Los códigos (P, N, F, RN) son los del acuerdo-compromiso. La matriz completa de trazabilidad está en el README.

Carpeta	Usuario	Qué hace y qué verifica
0. Auth	todos	Pide el token de `cliente1`, `despachante1`, `repartidor1`, `repartidor2` y `admin1` y los guarda en el entorno.
1. Datos maestros	`admin1`	Consulta centros, vehículos y repartidores de la semilla, y hace alta, modificación y baja de un vehículo (F05).
2. Cliente	`cliente1`	P1: perfil y alta de 5 pedidos en el centro 1: urgente, normal, con frío, uno para cancelar y uno fuera de cobertura. Todos quedan RECIBIDO.
3. Planificación	`despachante1`	P2: planifica el día. Verifica que el urgente y el normal queden en la misma ruta, que el pedido con frío vaya en el vehículo refrigerado y que el de fuera de cobertura quede NO_ASIGNABLE con su motivo (RN07-RN10). P6: reprograma el no asignable. P7: el cliente cancela un pedido ASIGNADO y la ruta se recalcula sin esa parada.
4. Ejecución	el repartidor de la ruta	P3: mi ruta, inicio y pedidos EN_CAMINO por el outbox. N4: no se puede cancelar en la calle. P4: parada ENTREGADA. P5: parada FALLIDA con motivo y ruta FINALIZADA.
5. Seguimiento	varios	Pedido ENTREGADO; el fallido vuelve a RECIBIDO con 1 intento (RN16). P8: historial completo con fecha y autor. Notificaciones sin errores. P9: distancia con su fuente.
6. Negativos y seguridad	varios	N1, N2, peso negativo, N4 a N8, N10 a N12: cada violación de regla o acceso indebido devuelve 400, 401, 403, 409 o 422, nunca 500.

> **Repartidor.** el planificador decide qué repartidor recibe la ruta principal. La carpeta 3 lo detecta y la carpeta 4 usa automáticamente el token de ese repartidor (`tokenRepartidorRuta`); el otro se usa para probar el acceso a una ruta ajena (N7).

5. Repetir la prueba

Vuelve las bases al estado inicial: borra pedidos, clientes, rutas y avisos, y conserva los centros, vehículos y repartidores de la semilla. Desde la carpeta del repositorio, con la contraseña del usuario `postgres`:

```
& "C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -f infra/reset-datos.sql
```

(Cambiar `18` por la versión instalada.) No hace falta reiniciar los servicios.

6. Escenarios que se prueban a mano

Requieren detener un servicio o mirar un buzón, así que no están en la colección.

6.1 Tolerancia a caídas (V7)

#	Acción	Resultado esperado
1	Correr la colección hasta la carpeta 4, request **P3 Iniciar ruta**.	Ruta EN_CURSO.
2	Detener pedidos-service (Ctrl+C en su terminal).	-
3	Ejecutar **P4 Parada ENTREGADA**.	**200** igual: despacho registra la entrega aunque pedidos esté caído.
4	Ejecutar **Notificaciones con error** cambiando `estado=ERROR` por `estado=PENDIENTE`.	Aparece el aviso pendiente.
5	Volver a levantar pedidos-service.	En menos de 15 s el aviso queda ENVIADA y el pedido pasa a ENTREGADO.

6.2 Dependencia caída al crear un pedido (N3)

Detener despacho-service y ejecutar **P1 Crear pedido NORMAL**: responde **503** y el pedido no se crea, porque no se puede validar el centro.

6.3 OSRM (distancias reales)

La request **P9 Distancia entre dos puntos** muestra en `fuente` de dónde salió el dato:

fuente	Significado
`OSRM`	Consultado a OSRM en este momento: distancia y tiempo reales por calle.
`CACHE`	Ya se había consultado y se reutiliza (vigencia de 30 días).
`CACHE_VENCIDA`	OSRM no respondió y se usó un dato viejo de la caché.
`ESTIMADA`	OSRM no disponible y sin caché: línea recta × 1,3 a 30 km/h. La ruta queda marcada `estimada`.

Para ver el respaldo en acción, levantar despacho-service con `OSRM_URL` apuntando a una dirección inexistente: la planificación se completa igual y las rutas salen con `estimada = true`.

6.4 Mails de cambio de estado

pedidos-service manda un mail al email del perfil del cliente en el alta y en cada cambio de estado. Para verlos, registrar el perfil con un email real y correr la colección. En el log de pedidos-service aparece `Mail enviado a ...` por cada uno; si dice `No se pudo enviar`, Gmail rechazó el login: `spring.mail.password` tiene que ser una contraseña de aplicación de Google (16 caracteres).

7. Problemas comunes

Síntoma	Causa y solución
Falla la carpeta 0 (`ECONNREFUSED` a 8180)	Keycloak no está corriendo o no importó el realm (sección 2, paso 2).
Fallan todas las demás (`ECONNREFUSED` a 8080)	El gateway no está corriendo.
401 en requests sueltas después de un rato	Los tokens duran 15 minutos. Volver a ejecutar la carpeta 0.
Falla **P2 Planificar el día** o lo que sigue	Ya se corrió hoy: los vehículos y repartidores del centro están usados. Correr el reset (sección 5).
503	Un microservicio no responde. Levantarlo y repetir.
Request suelta con un id vacío	Falta ejecutar antes la request que lo guarda (respetar el orden de las carpetas).